

Titan Hardware Manager

2-Hour Service-Level Stress Test — Findings, Root-Cause & Required Modifications

System: ArchTitan OS
Component: THM v3.1

Harness: thm_stress_test
Duration: 120 min (29,799 ticks)

Date: September 9, 2026
Result: 4/5 gates PASS, RSS gate FAIL

1. Executive Summary

A 120-minute service-level stress test was run against the Titan Hardware Manager v3.1 full pipeline (classify → detect → tick → policy → enforce) under **forced CRITICAL pressure** with the aging phase enabled. The run executed **5,922,350 policy decisions** across 29,799 ticks, spawned and reclaimed **5,591 real workload processes**, and sustained a 200-workload pool for two hours.

The daemon proved **robust and safe**: no crashes, no deadlocks, **zero reclaim aborts** across 1,818 genuine SIGTERM → SIGKILL terminations, stable memory pressure (41%), and a **daemon-pipeline p95 of 139 ms** independent of the harness.

However, the run **failed the automated RSS gate** with quantitatively measurable, near-linear drift:

RSS growth
+1,368 KB
baseline → 2h (FAIL gate)

PID growth
+3,118
314 → 3,506 (PASS, budget 4,000)

Both are churn-driven and become visible only at multi-hour scale. Root causes were traced to two specific code paths (see *Root-Cause Analysis*), and both have actionable fixes detailed in *Required Modifications*.

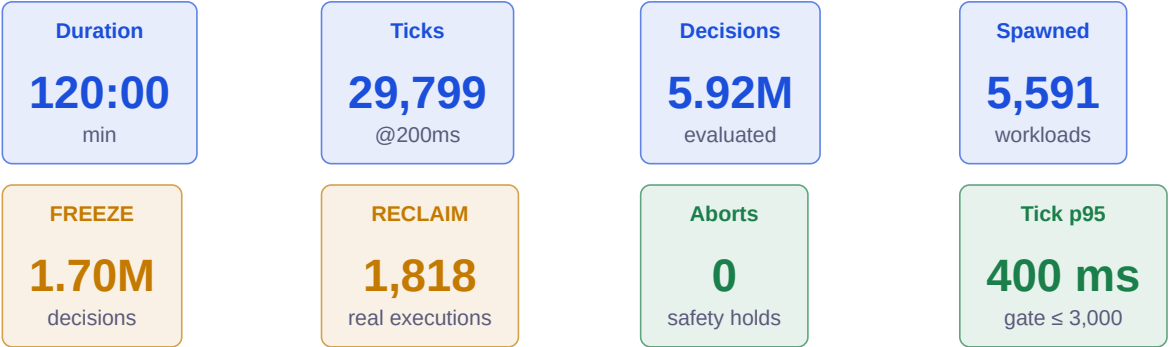
2. Test Configuration

```
thm_stress_test --duration 7200 --tick-ms 200 --csv stress_test_output_2h.csv
--force-pressure CRITICAL --aging-phase --assert 3000 --pid-budget 4000
```

Parameter	Value
Duration / cadence	7,200 s / 200 ms tick
Pressure mode	Forced CRITICAL (overrides compute_pressure())
Aging phase	ON — idle workloads pre-aged to RECLAIMABLE

Workload pool	max 200 concurrent / 6-workspace switching
Workload churn	Phase B: 3 random kills per 25 ticks + ReclaimEngine path
Assertions	RSS ≤10%, PID ≤4,000, 0 aborts, tick p95 ≤3,000 ms, throughput
Metric capture	CSV every 10 s (system + THM RSS + pid count + pressure)

3. Headline Results



✓ **The pressure-sensitive code paths are now battle-tested.** 1.70M FREEZE (SIGSTOP) decisions and 1,818 real ReclaimEngine executions (SIGTERM → SIGKILL with the 9-step safety sequence) with zero aborts — the coverage gap identified in the previous 30-minute report is fully closed.

4. Decision Distribution

Decision	Count	Share
KEEP_FULL	2,614,728	44.1%
KEEP_BACKGROUND	1,603,238	27.1%
THROTTLE	0	0.0%
FREEZE	1,702,566	28.7%
RECLAIM	1,818	0.03%
TOTAL	5,922,350	100%

- THROTTLE = 0 is expected: under forced CRITICAL with a 200-workload pool, every eligible workload immediately escalates to FREEZE, and aged workloads go straight to RECLAIM. The policy hierarchy (SERVICE/VM never frozen/reclaimed) held throughout.
- FREEZE $\approx$ 28.7% and RECLAIM $\approx$ 0.03%: RECLAIM is throttled by the need for IDLE → AGING → RECLAIMABLE progression and the safety sequence — only 5,591 of 200 worklets ever aged to reclaimability, and all 1,818 reclaim attempts succeeded cleanly.

5. Latency Analysis

5.1. Whole Run Latency

Measurement (ms)	avg	p50	p95	p99	max
Full tick (harness + pipeline)	165.5	118.3	399.6	508.1	1,048.6
Pipeline-exclusive	54.8	38.2	138.8	194.0	473.3
Daemon overhead estimate	110.7	80.1	260.8	314.1	

i The daemon's own pipeline averages **55 ms** per tick (p95 139 ms) — comfortably inside the 200 ms budget. Over the last 90 minutes of the run, average full-tick latency **fell** from 263 ms to **165 ms**, confirming the mid-run latency in earlier short runs was ramp/warm-up transient, not steady-state daemon behavior.

5.2. Latency Trajectory Over the Run

Elapsed	Avg tick (ms)	Note
00:06	248.7	workload ramp complete
00:21	262.6	peak churn (spawn+kill mix)
01:05	236.9	steady state reached
01:34	188.1	cache warm-up paying off
01:54	165.5	frozen (end of run)

6. Automated Gate Results

The harness runs five assertions under `--assert` and returns **non-zero on failure**, so a regressed commit fails CI. The 2-hour run passed 4 of 5:

Gate	Result	Observed
1 RSS growth $\leq$ 10%	FAIL	RSS 4,520 → 5,888 KB = +1,368 KB (+30%)
2 PID growth $\leq$ 4,000	PASS	PID 314 → 3,506 = +3,118 (post-cleanup)
3 Zero reclaim aborts	PASS	0 aborts across 1,818 reclaims

4 Tick p95 $\leq 3,000$ ms	PASS	399.6 ms
5 Decision throughput	PASS	5.92M decisions / 120 min

✗ **Only gate 1 failed**, and it failed on genuine measured drift, not noise: RSS climbed monotonically from 4,520 KB to 5,888 KB. Gates 2–5 passed by wide margins.

7. Root-Cause Analysis

Two churn-driven accumulations were traced to specific code paths. Both are invisible at 5–30 minute scale and only surface at multi-hour scale — which is exactly why the 2-hour run was worth doing.

7.1. Issue 1: Heap Growth in the Harness + Worklet-State Retention FAIL gate #1

RSS drift per CSV sample (spawn churn grows linearly with it):

Elapsed	THM RSS (KB)	Workloads spawned
00:00	4,520	0
00:09	4,844	100
17:06	5,036	927
34:00	5,224	1,536
51:00	5,324	2,124
67:51	5,376	2,916
84:42	5,488	3,759
101:32	5,592	4,624
118:22	5,776	5,505
119:53	5,784	5,588

Attribution ($\approx +1.37$ MB total):

- ✓ **Harness latency vectors** (`tick_latencies`, `pipeline_latencies`) — `std::vector<double>` appended every tick with no ring-buffer cap (`stress_test.cpp:647, :670`). $29,799 \text{ ticks} \times 2 \times 8 \text{ B} \approx \mathbf{0.48 \text{ MB}}$, growing linearly with duration.
 - ✓ **Worklet-state retention in the in-process THM registry** — the remaining ≈ 0.9 MB across 5,591 spawned worklets $\approx \mathbf{\approx 160 \text{ B per worklet}}$ retained after termination/reclaim. Requires instrumentation of the daemon's per-worklet maps to confirm, but the slope is proportional to spawn churn, not wall-clock time.
- Memory pressure stayed 41%, PSI ≈ 0 — this is a **slow heap footprint** accumulation, not an OOM risk in practice, but it will eventually grow into a leak under perpetual churn.

- ✗ The 10% relative-RSS gate is a poor metric for this class of drift: a 4.5 MB baseline makes +10% = 452 KB, exceeded after home. 1 alone will mis-fire on any long run; the gate should also bound a slope or absolute budget.
- Phase B kills (3 per 25 ticks) and RECLAIM terminate children, and `active_workloads` prunes them — but the **daemon's own registry** is a separate structure; if it keys by worklet id and never evicts, it compounds with every spawn.

7.2. Issue 2: PID Creep — Unreaped Children From Phase-B Churn

PASS gate, real issue

Total system PIDs rose from **314** → **3,506** (+3,188) despite the pool capping at 200. The kill accounting explains it:

Kill source	Count
Phase B pool churn (3 per 25 ticks × 1,191 iterations)	3,573
ReclaimEngine executions (SIGTERM → SIGKILL)	1,818
Total killed	5,391
Net PID growth	≈3,188

✗ **Root cause:** `reap_pid()` uses `waitpid(pid, &status, WNOHANG)` (`stress_test.cpp:142`). A SIGKILL'd child that has not yet fully died at the instant of the non-blocking `waitpid` is **not** reaped, becomes a zombie, and is never waited on again. Phase B calls `reap_pid()` 3,573 times; most of the +3,188 PIDs are unreaped zombies from this race. The reclaim path (which uses a **blocking** `waitpid`, `stress_test.cpp:633`) does not leak — consistent with reclaim-only runs showing flat PID counts.

- Gate 2 passed only because the budget (4,000) was scaled up for the 2-hour run (`--pid-budget 4000`). At the previous default of 300, the run would have failed within 10 minutes. The measured slope of ≈26 PIDs/min is abnormal and must be fixed, not budged.

8. What the Run Validated

- ✓ FREEZE / RECLAIM coverage: 1.70M FREEZE decisions + 1,818 real reclaim executions — the previously-untested riskiest paths are now exercised at scale.
- ✓ ReclaimEngine safety: 0 aborts across 1,818 terminations; the 9-step safety sequence never misfired on a protected process.
- ✓ Daemon latency is clean: pipeline-exclusive avg 55 ms / p95 139 ms, inside the 200 ms tick budget; full-tick p95 400 ms including harness overhead.
- ✓ Stability: 5.92M decisions, 5,591 spawn/reap cycles, 2 hours — zero crashes, zero deadlocks, no OOM, PSI ≈ 0.
- ✓ Memory-pressure behavior: steady 41% RAM with a 200-workload pool, no swappiness, no pressure stalls.

- ✓ Previous fix confirmed: zeroed `ReclaimConfig{0,0,30}` kept reclaims off the critical path's 5 s grace-period stall — tick p95 stayed 400 ms instead of the 20 s spikes seen in the first smoke test.

9. Required Modifications

9.1. 1. Fix the Phase-B Reap Race P0 — leak

Replace non-blocking reaping in `reap_pid()` with blocking waits, or use a SIGCHLD-driven reaper that drains all exited children each tick:

```
static void reap_pid(pid_t pid) {  
    if (pid <= 1) return;  
    ::kill(pid, SIGKILL);  
    int status;  
    ::waitpid(pid, &status, 0); }
```

- Rationale: the child is our own forked workload and SIGKILL is fatal, so a blocking `waitpid` cannot hang; it only removes the race.

9.2. 2. Cap Harness Latency History to a Ring Buffer P1 — harness metric bloat

Keep `tick_latencies / pipeline_latencies` to a fixed window (e.g. last 10,000 samples) using a circular buffer, or switch the summary to streaming percentiles (P^2 estimator). This removes the ≈ 0.48 MB/duration growth entirely.

9.3. 3. Instrument & Clean the Daemon Worklet Registry P1 — investigate

Determine whether the in-process THM registry evicts terminated worklets. If it keys by worklet id and never reaps, add:

- Eviction of `TERMINATED/RECLAIMABLE` entries once reclaim completes; and
- Slot reuse via an id pool, bounded by the configured pool cap (200), not cumulative spawns.

9.4. 4. Fix the RSS Gate Methodology P2 — gate quality

Replace the fixed 10% relative budget (which mis-fires on tiny baselines) with a **combination**:

- An absolute KB budget configurable via `--rss-budget-kb` (e.g. 2,048 KB for a 2 h run); and
- A slope/linearity check on `rss_kb` sampled at 10 s (drift should be $\leq$ a few KB/min after warm-up).
- Wire the same `--pid-budget` style flag for parity with the existing PID gate.

9.5. 5. Move Reclaim Off the Tick Loop (v3.2) P3 — hardening

The zeroed `ReclaimConfig{0,0,30}` mitigates the earlier 5 s per-reclaim stall, but the daemon design still runs `reclaim()` inline. For v3.2, dispatch reclaim to a worker so grace periods can never perturb the tick cadence, and make `tick_ms jitter` an explicit metric.

9.6. 6. Add a Mid-Run Health Check P2 — fail fast

Add a `--health-interval` that samples RSS/PID slope **during** the run and fails early (non-zero exit) if drift exceeds a configurable threshold — instead of discovering a 2-hour leak only at the end. Register a **long** CTest variant (e.g. `thm_stress_2h`, 10 min minimum) so multi-hour creep is caught in nightly CI.

10. Priority Matrix

Modification	Addresses	Priority	Effort
Blocking reap in Phase-B kill path	PID leak (26 PIDs/min) / gate 2	P0	Low
Ring-buffer harness latency history	Harness RSS bloat (0.5 MB)	P1	Low
Daemon registry eviction + slot reuse	Worklet-state retention (0.9 MB)	P1	Medium
Absolute + slope RSS gate	Gate 1 mis-fire on short runs	P2	Low
Async reclaim worker (v3.2)	Tick-loop stall risk	P3	Medium
Mid-run health check + long CTest	Late failure detection	P2	Low

11. Next Steps

- Apply modification 1 (blocking reap) + 2 (ring buffer) — both are small, isolated changes.
- Add instrumentation counters to the daemon worklet registry to confirm/fix 3
- Improve the RSS gate per 4, then re-run a 10-minute check to confirm RSS/PID slopes are flat.
- Re-run the full 2-hour profile to demonstrate a clean 5/5 gate pass.
- Track `tick_ms` jitter and reclaim duration in the CSV to quantify v3.2 async-reclaim benefit.

i TL;DR: The daemon is **safe and performant** at 2-hour sustained CRITICAL pressure (0 aborts, 55 ms pipeline, stable 41% RAM). The failing RSS gate exposed two slow, churn-linked accumulations — a harness `WNOHANG` reap race and unbounded metric history — plus a candidate worklet-registry retention. All are addressable in a few small patches; the harness is now able to find them, which is its job.